

p-sheetcheck

static analysis for financial spreadsheets

Project Managers

Maadhavan Gupta(PO '28)

Aneesh Raghavan(PO '28)

Overview

Financial spreadsheets are used to make high-stakes decisions in consulting, finance, and business, but many spreadsheet mistakes are not syntax errors. A formula can be perfectly valid in Excel while still referencing the wrong year, using the wrong operator, breaking a repeated pattern, or replacing a formula with a hardcoded value. These errors are especially difficult to catch in large models with hundreds or thousands of linked cells.

p-sheetcheck will treat a spreadsheet more like a computer program than a table of numbers. We will parse formulas into structured representations, build a dependency graph between cells, and infer repeated computational patterns across a workbook. The system will then flag cells whose structure or dependencies differ in suspicious ways from the surrounding model.

The main research question is: can we infer enough of a spreadsheet's intended computational structure to detect semantic formula errors without knowing the correct output in advance? The business setting gives us a useful and realistic test case, but the core of the project is computer science: parsing, program analysis, graph algorithms, anomaly detection, and empirical evaluation.

Research Questions

- Can formulas be normalized into structural signatures that remain comparable even when copied across different rows, columns, or sheets?
- Can repeated local patterns and dependency-graph structure be used to infer invariants in a spreadsheet automatically?
- Which classes of errors are best detected by local formula patterns, graph-based features, or a combination of both?
- How well do these methods generalize to financial models that were not seen when the detector was developed?

Goals

- Build a parser that extracts formulas, cell references, and workbook structure from Excel files and converts formulas into a normalized abstract representation.
- Construct a directed dependency graph for each workbook and develop methods for identifying repeated computational structure.
- Implement multiple error detectors, starting with simple baselines and progressing to structural and graph-based methods.
- Create an evaluation benchmark by introducing controlled formula mutations into real or carefully constructed financial models.
- Measure precision, recall, F1 score, false-positive rate, and runtime, and compare our methods against simpler heuristics.

- If time permits, add a lightweight interface that highlights suspicious cells and explains why each one was flagged.

Tech Stack

- Python for the core analysis pipeline.
- openpyxl for workbook/formula extraction; a formula parser or small custom grammar for converting formulas into AST-like representations.
- NetworkX initially for dependency-graph construction and graph experiments, with custom graph routines where needed.
- pandas / NumPy for benchmark generation and analysis; pytest for automated testing.
- Git/GitHub for version control and code review; Docker if useful for reproducible setup.
- Optional stretch: FastAPI + React/TypeScript for an interactive spreadsheet review interface.

Member Requirements

We plan to recruit 5-6 members, with an expected commitment of about 3-4 hours per week outside the weekly meeting. Prior programming experience is helpful but not required for every role. Members with Python/data-structures experience can work on parsing, graphs, and detection algorithms; members with less programming experience can begin with spreadsheet corpus collection, error taxonomy, benchmark construction, testing, and evaluation, then move into paired implementation tasks as the semester progresses.

We want the team to be technically ambitious without making the project inaccessible. The first two weeks will include onboarding on spreadsheet formulas, Python tooling, graphs, Git, and our evaluation setup so that everyone can contribute to a shared codebase.

Timeline

Week	Focus
9/8-9/14	Scope the problem; collect example financial models; define error taxonomy and benchmark format.
9/15-9/21	Onboarding: Python/Git, spreadsheet internals, formula references; build workbook ingestion pipeline.
9/22-9/28	Formula parsing and normalization; represent formulas using relative references / AST-style structures.
9/29-10/5	Build workbook dependency graphs; visualize and sanity-check graph structure.
10/6-10/12	Implement baseline detectors: neighbor consistency, formula-frequency, and simple reference-distance heuristics.
10/13-10/19	Develop structural-pattern / invariant inference; initial experiments on real workbooks.
10/20-10/26	Develop graph-based detectors and combine local + dependency features.
10/27-11/2	Build automated mutation engine to inject realistic spreadsheet errors at scale.
11/3-11/9	Run benchmark experiments; measure precision, recall, F1, false positives, and runtime.
11/10-11/16	Error analysis and iteration; identify which bug classes remain difficult and improve methods.
11/17-11/23	Stretch work: unit/type inference or lightweight reviewer UI; finalize technical write-up structure.
11/24-11/30	Thanksgiving / lighter week; documentation, cleanup, and reproducibility checks.

Week	Focus
12/1-12/7	Final experiments, demo, presentation, and short research-style report.

What Success Looks Like

A successful semester does not require us to solve every spreadsheet error. Our target is to end with a reproducible analysis system, a clearly defined benchmark, and an honest comparison between several detection approaches. The ideal final result is a small research-style finding: for example, showing that structural and dependency information catches a meaningful class of errors that simpler spreadsheet heuristics miss.